Server Security Hardening Guide

CakePHP API + Vue SPA on DigitalOcean

with Cloudflare Protection

Prepared: March 2026

Stack: Ubuntu 24.04 LTS · Nginx · PHP 8.x · CakePHP 4/5 · Vue 3 · MySQL/PostgreSQL

Table of Contents

1. SSH Hardening

SSH is the primary attack vector for any public-facing server. These steps reduce your exposure dramatically.

1.1 Create a Non-Root User

Never run services or log in as root. Create a dedicated user with sudo access.

```
# Create user and add to sudo group
adduser deployer
usermod -aG sudo deployer

# Copy SSH keys to the new user
rsync --archive --chown=deployer:deployer ~/.ssh /home/deployer
```

1.2 Disable Root Login and Password Authentication

Edit /etc/ssh/sshd_config to lock down SSH access.

```
sudo nano /etc/ssh/sshd_config

# Change or add these directives:
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
ChallengeResponseAuthentication no
UsePAM yes
```

Restart SSH to apply:

```
sudo systemctl restart sshd
```

> **Warning**
> Before restarting SSH, verify you can log in as your new user with your SSH key in a SEPARATE terminal session. Otherwise you risk locking yourself out.

1.3 Change the Default SSH Port

Moving SSH off port 22 reduces automated scan noise significantly.

```
# In /etc/ssh/sshd_config change:
Port 2222

# Restart SSH
sudo systemctl restart sshd

# Connect using the new port going forward:
ssh -p 2222 deployer@your-server-ip
```

1.4 Install and Configure Fail2Ban

Fail2Ban automatically bans IPs that show malicious signs (repeated failed logins).

```
sudo apt update && sudo apt install -y fail2ban

# Create a local config (never edit the main jail.conf)
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

Add/modify the [sshd] section:

```
[sshd]
enabled = true
port = 2222
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 3600
findtime = 600
```

```
sudo systemctl enable fail2ban
sudo systemctl start fail2ban

# Verify it's running:
sudo fail2ban-client status sshd
```

2. Firewall Configuration (UFW)

The goal is to only allow SSH from anywhere (on your custom port) and HTTP/HTTPS exclusively from Cloudflare IP ranges, so nobody can bypass Cloudflare by hitting your origin IP directly.

2.1 Basic UFW Setup

```
sudo apt install -y ufw

# Default policies
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow SSH on custom port
sudo ufw allow 2222/tcp comment 'SSH'
```

2.2 Allow Only Cloudflare IPs for HTTP/HTTPS

Cloudflare publishes their IP ranges. Create a script that fetches them and creates UFW rules:

```
#!/bin/bash
# File: /usr/local/bin/update-cloudflare-ips.sh

# Remove old Cloudflare rules
sudo ufw status numbered | grep 'Cloudflare' | \
  awk -F'[][]' '{print $2}' | sort -rn | \
  while read num; do sudo ufw --force delete $num; done

# Add current Cloudflare IPv4 ranges
for ip in $(curl -s https://www.cloudflare.com/ips-v4); do
  sudo ufw allow from $ip to any port 80,443 proto tcp \
    comment 'Cloudflare'
done

# Add current Cloudflare IPv6 ranges
for ip in $(curl -s https://www.cloudflare.com/ips-v6); do
  sudo ufw allow from $ip to any port 80,443 proto tcp \
    comment 'Cloudflare'
done

sudo ufw reload

sudo chmod +x /usr/local/bin/update-cloudflare-ips.sh
sudo /usr/local/bin/update-cloudflare-ips.sh
```

2.3 Enable UFW

```
sudo ufw enable
sudo ufw status verbose
```

Tip

Add a cron job to update Cloudflare IPs weekly: sudo crontab -e and add 0 3 * * 1 /usr/local/bin/update-cloudflare-ips.sh

2.4 Enable Automatic Security Updates

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

This ensures critical security patches are applied automatically.

3. Nginx Hardening

Your web server is the front door. These headers and configurations prevent common attacks and information leakage.

3.1 Hide Server Information

```
# In /etc/nginx/nginx.conf, inside the http {} block:
server_tokens off;
more_clear_headers Server;
```

3.2 Security Headers

Add these inside your server {} block or in a shared snippet:

```
# /etc/nginx/snippets/security-headers.conf

# Prevent MIME-type sniffing
add_header X-Content-Type-Options "nosniff" always;

# Prevent clickjacking
add_header X-Frame-Options "SAMEORIGIN" always;

# XSS protection (legacy browsers)
add_header X-XSS-Protection "1; mode=block" always;

# Enforce HTTPS for 1 year + subdomains
add_header Strict-Transport-Security
  "max-age=31536000; includeSubDomains; preload" always;

# Referrer policy
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

# Permissions policy
add_header Permissions-Policy
  "camera=(), microphone=(), geolocation=()" always;

# Content Security Policy (adjust to your needs)
add_header Content-Security-Policy
  "default-src 'self'; script-src 'self'; style-src 'self'
   'unsafe-inline'; img-src 'self' data: https:;
   connect-src 'self' https://api.yourdomain.com;
   font-src 'self'; frame-ancestors 'none';" always;
```

Include the snippet in your server blocks:

```
include /etc/nginx/snippets/security-headers.conf;
```

3.3 Force HTTPS and Redirect

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    return 301 https://$host$request_uri;
```

```
}
```

3.4 Rate Limiting

Add rate limiting zones in the http {} block and apply to API locations:

```
# In nginx.conf http {} block:
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=login:10m rate=3r/m;

# In your server {} block:
location /api/ {
    limit_req zone=api burst=20 nodelay;
    # ... proxy/fastcgi config ...
}

location /api/users/login {
    limit_req zone=login burst=5 nodelay;
    # ... proxy/fastcgi config ...
}
```

3.5 SSL Configuration (Origin Certificate)

Even with Cloudflare in front, you need a valid cert on your origin for Full (Strict) mode. Use a Cloudflare Origin Certificate:

```
# Generate Origin Certificate in Cloudflare Dashboard:
# SSL/TLS > Origin Server > Create Certificate

# Save the cert and key:
sudo mkdir -p /etc/ssl/cloudflare
sudo nano /etc/ssl/cloudflare/origin.pem    # paste certificate
sudo nano /etc/ssl/cloudflare/origin.key     # paste private key
sudo chmod 600 /etc/ssl/cloudflare/origin.key

# In your Nginx server block:
ssl_certificate /etc/ssl/cloudflare/origin.pem;
ssl_certificate_key /etc/ssl/cloudflare/origin.key;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers on;

# Test and reload Nginx:
sudo nginx -t && sudo systemctl reload nginx
```

4. Cloudflare Configuration

You already have Cloudflare in place. Here are the settings to verify and enable for maximum protection.

4.1 SSL/TLS Mode

Go to SSL/TLS in your Cloudflare dashboard and set encryption mode to Full (Strict). This ensures the connection between Cloudflare and your origin is encrypted and validated.

4.2 WAF (Web Application Firewall)

- Enable Cloudflare Managed Ruleset under Security > WAF.
- Enable OWASP Core Ruleset for SQL injection, XSS, and other common attacks.
- Monitor the firewall events log for the first week and whitelist any false positives.

4.3 Bot Fight Mode

Enable under Security > Bots > Bot Fight Mode. This blocks known bad bots and challenges suspicious ones.

4.4 Rate Limiting Rules

Create rate limiting rules under Security > WAF > Rate Limiting Rules:

- API endpoints: Limit to 100 requests per minute per IP for /api/* paths.
- Login endpoint: Limit to 5 requests per minute per IP for /api/users/login.
- Action: Block or challenge when threshold exceeded.

4.5 Page Rules / Configuration Rules

- Cache Level: Standard for static assets (JS, CSS, images).
- Browser Cache TTL: Set to 4 hours for static assets.
- Disable caching for /api/* paths entirely.

4.6 Additional Settings

- Enable Always Use HTTPS under SSL/TLS > Edge Certificates.
- Enable HSTS in Cloudflare (mirrors your Nginx HSTS header).
- Set Minimum TLS Version to 1.2.
- Enable Authenticated Origin Pulls for extra origin validation.

5. CakePHP API Security

Your API layer is where most application-level vulnerabilities live. CakePHP has strong built-in security features that must be properly configured.

5.1 Disable Debug Mode in Production

```
// config/app_local.php
'debug' => false,

// Also ensure error reporting doesn't leak info:
'Error' => [
    'errorLevel' => E_ALL & ~E_DEPRECATED,
    'exceptionRenderer' => 'App\Error\AppExceptionRenderer',
],
```

Critical
Never deploy with debug = true. Stack traces expose file paths, database credentials, and internal application structure to attackers.

5.2 CORS Configuration

Install and configure the CORS middleware so only your Vue SPA domain can access the API:

```
// In src/Application.php -> middleware() method:
$middlewareQueue
    ->add(new CorsMiddleware([
        'origin' => ['https://yourdomain.com'],
        'methods' => ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
        'headers' => [
            'Accept',
            'Content-Type',
            'Authorization',
            'X-CSRF-Token',
        ],
        'credentials' => true,
        'maxAge' => 3600,
    ]));
```

5.3 CSRF Protection

Add CakePHP's CSRF middleware for state-changing requests:

```
// In src/Application.php -> middleware():
use Cake\Http\Middleware\CsrfProtectionMiddleware;

$csrf = new CsrfProtectionMiddleware([
    'httponly' => true,
    'secure' => true,
    'samesite' => 'Strict',
]);

$middlewareQueue->add($csrf);
```

> **Note for SPA**
> For token-based (JWT) auth, you may skip CSRF if tokens are sent in headers (not cookies). If using cookie-based sessions, CSRF is essential. Your Vue app must read the CSRF token from the cookie and include it in the X-CSRF-Token header.

5.4 Authentication & Authorization

```
composer require cakephp/authentication
composer require cakephp/authorization
```

Configure the Authentication plugin in your Application.php:

```
// In src/Application.php -> bootstrap():
$this->addPlugin('Authentication');
$this->addPlugin('Authorization');

// In getAuthenticationService():
$service = new AuthenticationService();
$service->loadIdentifier('Authentication.Password', [
    'fields' => ['username' => 'email', 'password' => 'password'],
]);

// Use JWT for API:
$service->loadAuthenticator('Authentication.Jwt', [
    'returnPayload' => false,
]);
```

5.5 Input Validation

Always validate and sanitize in your Table classes. CakePHP's ORM uses parameterized queries by default, but watch for raw SQL:

```
// SAFE - ORM uses prepared statements:
$users = $this->Users->find()
    ->where(['email' => $this->request->getData('email')])
    ->first();

// DANGEROUS - raw queries with user input:
// NEVER do this:
// $conn->execute("SELECT * FROM users WHERE
//   email = '" . $input . "'");

// If raw queries are needed, use bindings:
$conn->execute(
    'SELECT * FROM users WHERE email = :email',
    ['email' => $input],
    ['email' => 'string']
);
```

5.6 Secure Cookie Settings

```
// config/app.php -> Session:
'Session' => [
    'defaults' => 'php',
    'ini' => [
```

```
        'session.cookie_httponly' => true,
        'session.cookie_secure' => true,
        'session.cookie_samesite' => 'Strict',
    ],
],
```

6. Vue SPA Security

The frontend carries its own risks, mainly around XSS and token handling.

6.1 Token Storage

Never store JWT tokens or sensitive data in localStorage. It is accessible to any JavaScript running on the page, making it vulnerable to XSS.

Recommended approach: Use httpOnly cookies set by the API. The browser sends them automatically and JavaScript cannot read them.

```
// Backend sets the token as a cookie:
setcookie('token', $jwt, [
    'httponly' => true,
    'secure' => true,
    'samesite' => 'Strict',
    'path' => '/',
]);

// Vue axios config (sends cookies automatically):
axios.defaults.withCredentials = true;
```

6.2 Prevent XSS

- Never use v-html with user-supplied content.
- If v-html is necessary, sanitize with DOMPurify first.
- Use {{ }} interpolation (auto-escaped) instead of v-html wherever possible.

```
// Install DOMPurify:
npm install dompurify

// Usage in a component:
import DOMPurify from 'dompurify';
const cleanHtml = DOMPurify.sanitize(rawHtml);
```

6.3 Production Build Hardening

```
// vite.config.js (or vue.config.js)
export default defineConfig({
  build: {
    sourcemap: false,  // DO NOT expose source maps
  },
});
```

Also audit npm dependencies regularly:

```
npm audit
npm audit fix
```

6.4 Environment Variables

Never commit .env files with secrets. Use .env.example as a template and keep .env in .gitignore.

```
# .env.production
VITE_API_URL=https://api.yourdomain.com

# NEVER put secrets in VITE_ variables.
# They are embedded in the built JS and visible to anyone.
```

7. Database Security

Your database holds the most valuable data. Lock it down tight.

7.1 Bind to Localhost Only

For MySQL:

```
# /etc/mysql/mysql.conf.d/mysqld.cnf
bind-address = 127.0.0.1

sudo systemctl restart mysql
```

For PostgreSQL:

```
# /etc/postgresql/16/main/postgresql.conf
listen_addresses = 'localhost'

sudo systemctl restart postgresql
```

7.2 Create a Dedicated Application User

Never use the root database user for your application.

```
-- MySQL:
CREATE USER 'cakeapp'@'localhost'
  IDENTIFIED BY 'strong_random_password_here';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON your_database.* TO 'cakeapp'@'localhost';
FLUSH PRIVILEGES;

-- PostgreSQL:
CREATE USER cakeapp WITH PASSWORD 'strong_random_password';
GRANT SELECT, INSERT, UPDATE, DELETE
  ON ALL TABLES IN SCHEMA public TO cakeapp;
```

Important

The application user should NOT have DROP, ALTER, CREATE, or GRANT privileges. If you need migrations, use a separate migration user with broader permissions and run migrations manually or via CI/CD.

7.3 Backups

```
# Automated daily backup script
#!/bin/bash
# /usr/local/bin/db-backup.sh

BACKUP_DIR=/var/backups/mysql
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
mkdir -p $BACKUP_DIR

mysqldump -u backup_user -p'password' your_database \
  | gzip > $BACKUP_DIR/db_$TIMESTAMP.sql.gz
```

```
# Keep only last 30 days
find $BACKUP_DIR -name '*.sql.gz' -mtime +30 -delete

# Add to crontab (runs daily at 2 AM):
sudo crontab -e
0 2 * * * /usr/local/bin/db-backup.sh
```

Offsite Backups

Consider syncing backups to DigitalOcean Spaces or another S3-compatible service. A backup only on the same server is not a real backup.

8. Monitoring & Logging

Security without monitoring is incomplete. You need to know when something goes wrong.

8.1 Log Monitoring with Logwatch

```
sudo apt install -y logwatch

# Get a daily summary emailed to you:
sudo nano /etc/logwatch/conf/logwatch.conf
# Set: MailTo = your@email.com
# Set: Detail = Med
# Set: Range = yesterday
```

8.2 SSH Login Alerts

Create a script that sends you a notification on every SSH login:

```
# /etc/profile.d/ssh-alert.sh
if [ -n "$SSH_CLIENT" ]; then
  IP=$(echo $SSH_CLIENT | awk '{print $1}')
  echo "SSH login: $(whoami) from $IP at $(date)" \
    | mail -s "SSH Alert: $(hostname)" your@email.com
fi
```

8.3 CakePHP Error Logging

Ensure CakePHP logs errors to a file you can monitor:

```
// config/app.php -> Log:
'Log' => [
    'error' => [
        'className' => 'Cake\Log\Engine\FileLog',
        'path' => LOGS,
        'file' => 'error',
        'levels' => ['warning', 'error', 'critical', 'emergency'],
    ],
],
```

8.4 DigitalOcean Monitoring

- Enable the built-in DigitalOcean monitoring agent from your droplet's dashboard.
- Set up alerts for CPU > 80%, Memory > 90%, and Disk > 85%.
- Consider adding Uptime Robot or Better Uptime for HTTP monitoring of your Vue app and API endpoints.

9. Post-Hardening Checklist

Use this checklist to verify all measures have been applied.

Area	Item	Done?
SSH	Non-root user created with sudo access	☐
	Root login disabled in sshd_config	☐
	Password authentication disabled	☐
	SSH port changed from 22	☐
	Fail2Ban installed and active	☐
Firewall	UFW enabled with deny-all default	☐
	HTTP/HTTPS restricted to Cloudflare IPs only	☐
	Cloudflare IP update script scheduled via cron	☐
	Automatic security updates enabled	☐
Nginx	Server tokens hidden	☐
	All security headers configured	☐
	HTTPS enforced with redirect	☐
	Rate limiting on API and login routes	☐
	SSL/TLS with Cloudflare Origin Certificate	☐
Cloudflare	SSL mode set to Full (Strict)	☐
	WAF Managed Rules enabled	☐
	Bot Fight Mode enabled	☐
	Rate limiting rules created	☐
	Always Use HTTPS enabled	☐
	Minimum TLS version set to 1.2	☐
CakePHP	Debug mode OFF in production	☐
	CORS restricted to SPA domain only	☐
	CSRF protection enabled (if using cookies)	☐
	Authentication & Authorization plugins configured	☐

Area	Item	Done?
	All user input validated and sanitized	☐
	Cookie settings: httpOnly, secure, SameSite=Strict	☐
Vue SPA	No tokens stored in localStorage	☐
	v-html usage audited and sanitized	☐
	Source maps disabled in production	☐
	npm audit run with no critical vulnerabilities	☐
	No secrets in VITE_ environment variables	☐
Database	Bound to localhost only	☐
	Dedicated app user with minimal privileges	☐
	Automated backups with offsite copy	☐
Monitoring	Logwatch installed and configured	☐
	SSH login alerts enabled	☐
	CakePHP error logging configured	☐
	DigitalOcean alerts set up	☐